

Data Mining & Machine Learning
M.Sc. in Artificial Intelligence and Data Engineering
University of Pisa

Project Report

Academic Year 2025–2026

Project Title: HitPredictor-Steam: Game Success Prediction

Authors

Pedro Carneiro Junior — Student ID: 724961

Andrea Zanin — Student ID: 635350

Submission date: 24/07/2026

Abstract

We present HitPredictor-Steam, an end-to-end machine learning pipeline for predicting commercial video game adoption trajectories on Valve’s Steam platform using storefront metadata and community interaction endpoints. Operating on a consolidated dataset of 29,235 game records, we formulate success as a multi-class ordinal classification task across 5 ownership tiers (ranging from long-tail indie releases to AAA blockbusters). To evaluate pre-launch decision potential, the pipeline enforces strict bifurcation between Pre-Launch and Post-Launch operational modes to eliminate target leakage. Data hygiene routines parse technical specifications, clean script noise, and extract semantic text representations using TF-IDF and SentenceTransformers (‘all-mpnet-base-v2’). Class imbalance and power-law distribution challenges are mitigated inside cross-validation folds through dynamic majority undersampling, targeted SMOTENC oversampling strictly on the extreme minority class, and Tomek Links spatial boundary cleaning. Dimensionality reduction leverages Partial Least Squares Discriminant Analysis (PLS-DA). Candidate architectures were evaluated using a 5×3 Nested Cross-Validation setup. Hyperparameter optimization was conducted via Bayesian search (Optuna) and cached unified grid search, establishing XGBoost as the champion architecture. Post-hoc probability calibration using Differential Evolution optimized Quadratic Weighted Kappa (QWK), increasing overall test alignment to $QWK \approx 0.7026$ for the pre-release model. Model transparency is guaranteed through an inverse-scaled Explainable AI suite utilizing global SHAP attributions and local LIME case studies.

1. Introduction

The digital PC gaming market represents a multi-billion-dollar industry characterized by thousands of annual releases on storefront platforms like Steam. Commercial outcomes across this market adhere strictly to a steep power-law distribution: a tiny fraction of titles secure massive market adoption, while the overwhelming majority remain in the long-tail curve with minimal sales. Predicting a video game’s commercial adoption trajectory prior to release or early in its post-launch lifecycle allows independent developers, publishers, and venture investors to optimize marketing allocations, refine pricing strategies, and mitigate financial risk before committing capital.

Machine learning offers effective tools to model complex success dynamics from heterogeneous pre-release and post-release metadata, including pricing structures, system hardware thresholds, localization scope, and community interaction metrics. However, video game success prediction presents severe data mining challenges, including extreme power-law target imbalance, high-dimensional sparse feature spaces (e.g., user tags and textual descriptions), non-linear feature interactions, and potential target leakage between pre-launch and post-launch signals.

This project aims to:

1. Construct a robust end-to-end data hygiene pipeline to ingest, clean, and extract features from 29,235 raw Steam records while enforcing strict Pre-Launch vs. Post-Launch feature bifurcation to eliminate data leakage.
2. Mitigate severe power-law class imbalance inside cross-validation folds using dynamic

majority undersampling, SMOTENC minority synthesis, and Tomek Links spatial boundary pruning.

3. Implement supervised Partial Least Squares Discriminant Analysis (PLS-DA) alongside sentence transformer embeddings ('all-mpnet-base-v2') to compress high-dimensional textual metadata into discriminative latent components.
4. Evaluate candidate ensemble architectures via a rigorous 5×3 Nested Cross-Validation strategy, optimizing hyperparameters through Bayesian search (Optuna) and cached grid searches using Quadratic Weighted Kappa (QWK) as the primary ordinal metric.
5. Apply post-hoc probability calibration via Differential Evolution and provide complete model transparency using an inverse-scaled Explainable AI (XAI) suite incorporating SHAP and LIME frameworks.

2. Problem

We formulate video game success prediction on Steam as a supervised multi-class ordinal classification task across 5 ownership macro-tiers. The raw dataset consolidates 29,235 unique game records derived from official Steam Store endpoints (`steam_app_data.csv`) and SteamSpy ownership estimates (`steamspy_data.csv`). Raw SteamSpy ownership data spans 13 high-variance interval classes, with over 69% of games residing in the lowest adoption bracket. To enable robust supervised modeling, these intervals are engineered into 5 ordinal tiers:

- **Tier 0 (0 – 20,000):** Indie Long-Tail (Low adoption).
- **Tier 1 (20,000 – 100,000):** Healthy Niche (Sustainable indie).
- **Tier 2 (100,000 – 500,000):** Mid-Market Success (Breakout hits).
- **Tier 3 (500,000 – 2,000,000):** Major Success (AA level).
- **Tier 4 (2,000,000+):** Hit / AAA status.

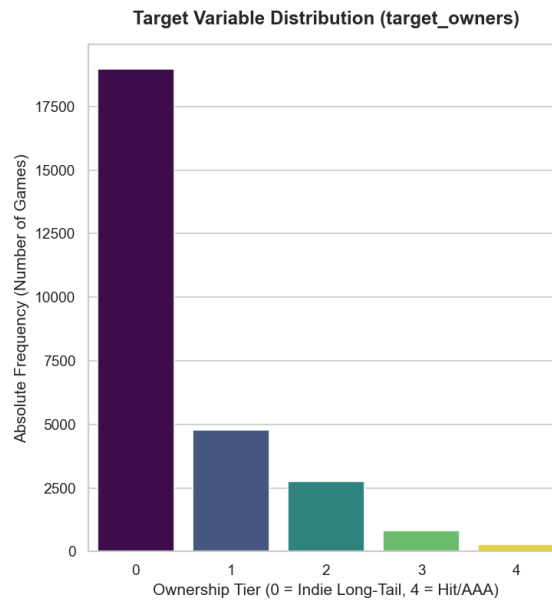


Figure 1. Class distribution.

Table 1 summarizes the problem setup, dataset dimensions, and validation protocols.

Table 1. Problem and dataset at a glance.

Attribute	Summary
Task type	Ordinal multi-class classification (5 Tiers: Tier 0 to Tier 4)
Dataset	HitPredictor-Steam dataset — 29,235 unique samples
Target variable	Ownership Tier (0: Long-Tail, 1: Niche, 2: Mid-Market, 3: Major, 4: AAA)
Features	Tabular metadata, hardware specs, TF-IDF tags, SentenceTransformer embeddings
Operational Modes	Pre-Launch Mode (Day-zero metadata) / Post-Launch Mode (With engagement)
Class distribution	Severe power-law skewed (Tier 0 dominates > 69%)
Evaluation metric(s)	Quadratic Weighted Kappa (QWK) [Primary], Macro F1, Accuracy, Recall
Validation Strategy	5×3 Nested Cross-Validation (5 Outer Folds, 3 Inner Folds)

2.1 Exploratory Data Analysis

Data profiling reveals structural target imbalance and high feature heterogeneity. Because post-release interaction indicators (e.g., average laytime, positive/negative reviews) are absent before launch, the pipeline enforces strict operational bifurcation:

- **Pre-Launch Mode:** Restricts input features exclusively to day-zero metadata (hardware requirements, language localizations, pricing, categories, developer history, and description text embeddings) to prevent target peeking.
- **Post-Launch Mode:** Retains engagement metrics (Metacritic score, playtime metrics, review counts) for post-release performance evaluation.

Missing numerical values occur primarily in system requirements due to unformatted HTML strings. regular expression routines parse minimum/recommended RAM specifications into standardized gigabytes (GB) and derive binary CPU/GPU tier flags (e.g., i5/i7, RTX/GTX). Descriptive tag vectors undergo normalization by filtering unparseable CJK and Cyrillic scripts. Missing continuous values undergo NaN median imputation, while boolean flags (e.g., controller support, DRM) are clipped to binary vectors.

Correlation and distribution analysis shows strong non-linear relationships. Higher ownership tiers (Tier 3 and Tier 4) exhibit significantly higher price points, broader language localizations, and higher hardware RAM baselines.

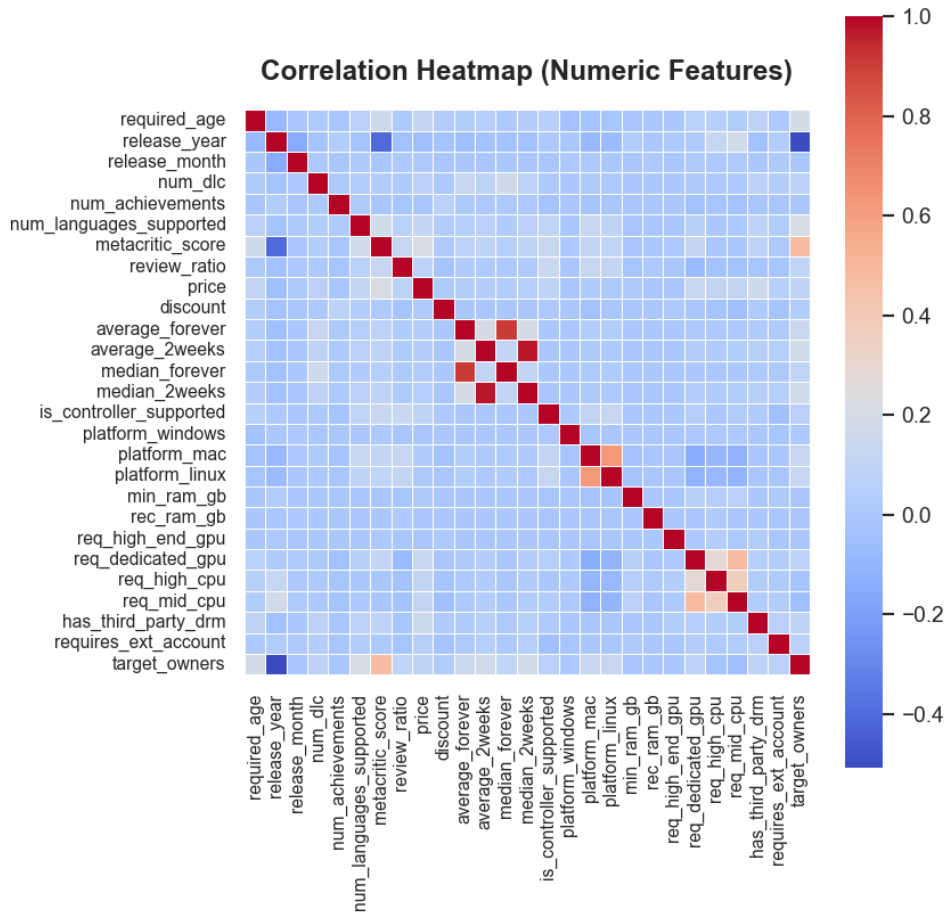


Figure 2. Correlation Heatmap

2.2 Feature Description

Features are organized into four major domain categories:

1. Commercial and Pricing Features:

- **price:** Base listing price in USD at launch. Measures product positioning and market tier.
- **num_languages_supported:** Total language localizations. Serves as a proxy for development scope and international reach.

2. Technical Specifications:

- **minimum_ram_gb / recommended_ram_gb:** Hardware memory baselines extracted via regex from HTML system requirement fields.
- **cpu_tier / gpu_tier:** Categorical and binary indicators representing hardware dependencies (e.g., RTX, GTX, Ryzen).

3. Creator and Storefront Metadata:

- **developer / publisher:** Developer and publisher historical output and categorical brand flags.
- **categories / tags:** Multi-label binary indicators representing Steam community tags and categories (e.g., Singleplayer, Co-op, Trading Cards).

4. Unstructured Text Representations:

- `description_embeddings`: Dense semantic vectors extracted from cleaned storefront descriptions using `SentenceTransformer('all-mpnet-base-v2')` and reduced via PLS-DA.
- `tfidf_features`: Sparse numerical vectors (1-2 grams) generated by applying Term Frequency-Inverse Document Frequency (TF-IDF) exclusively to the `short_description` metadata.

3. Pipeline and Experiments

We construct a modular execution pipeline to process raw storefront attributes, perform resampling, train candidate classifiers, and evaluate out-of-sample performance. Figure 3 outlines the overall architecture.

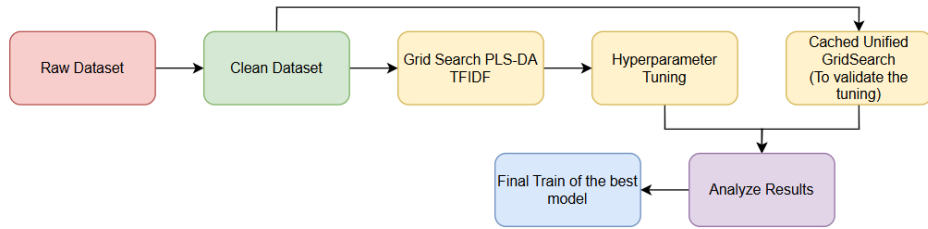


Figure 3. Project pipeline architecture.

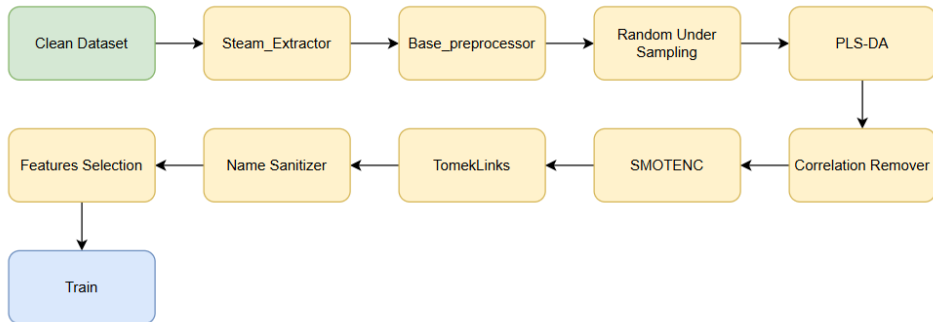


Figure 4. Data pre-processing pipeline.

3.1 Preprocessing and Data Hygiene

Preprocessing routines run deterministically within cross-validation folds to guarantee zero data leakage:

1. **Merging & Filtering:** Steam Store metadata and SteamSpy statistics are joined via `appid`. DLCs, non-game software, and malformed rows are dropped.
2. **Hardware Parsing:** HTML markup is stripped, and numeric RAM values are standardized into gigabytes. Binary hardware flags for CPU and GPU tiers are extracted through regex keyword matching.
3. **Text Normalization & Vectorization:** HTML tags and whitespace artifacts are removed from short descriptions and names. Textual fields are vectorized using TF-IDF and dense embeddings extracted via `SentenceTransformer('all-mpnet-base-v2')`.

3.2 Class Imbalance Resolution & Spatial Boundary Pruning

To resolve severe long-tail target imbalance, a multi-stage resampling sequence is executed strictly within outer training folds:

- **Dynamic Majority Undersampling:** Caps Tier 0 volume to 25% of its training fold size.
- **Targeted SMOTENC Oversampling:** Synthesizes instances exclusively for the extreme minority class (Tier 4). To prevent spatial noise and boundary blurring between adjacent tiers, the oversampling strategy dynamically anchors Tier 4’s volume to exactly 50% of Tier 3’s volume within each training fold, leaving all other classes untouched.
- **Tomek Links Boundary Pruning:** Cleans cross-class decision boundaries by removing overlapping instance pairs post-oversampling.

3.3 Supervised Dimensionality Reduction (PLS-DA)

To handle the high-dimensional semantic spaces generated by Sentence Transformers, Partial Least Squares Discriminant Analysis (PLS-DA) is integrated as a supervised dimensionality reduction transformer. PLS-DA directly projects the dense text embeddings extracted from the `detailed_description` into a compact set of latent directions ($n_{\text{components}} = 20$) that explicitly maximizes the covariance between the textual feature space and the 5 target ownership tiers.

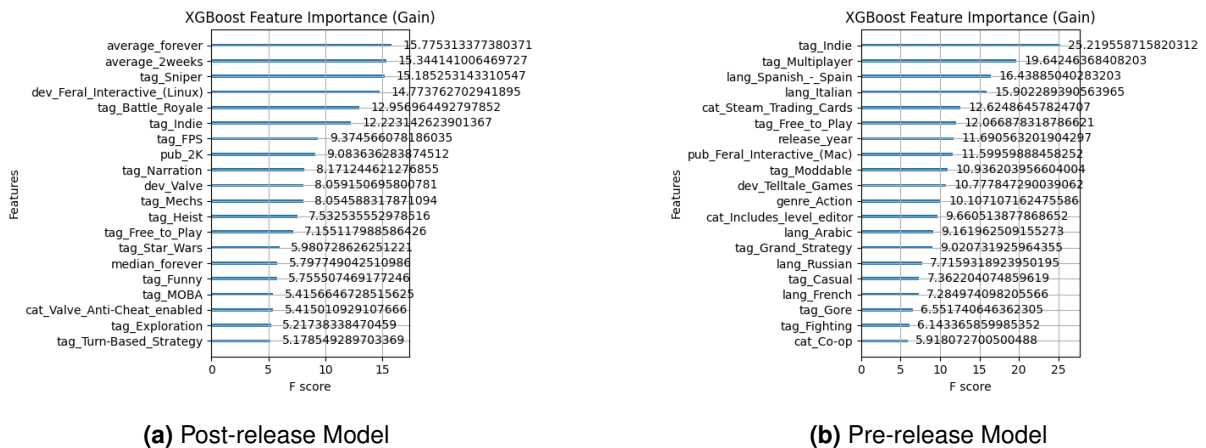


Figure 5. XGBoost Features Gain.

3.4 Model Selection and Nested Cross-Validation

Model evaluation utilizes a 5×3 Nested Cross-Validation strategy:

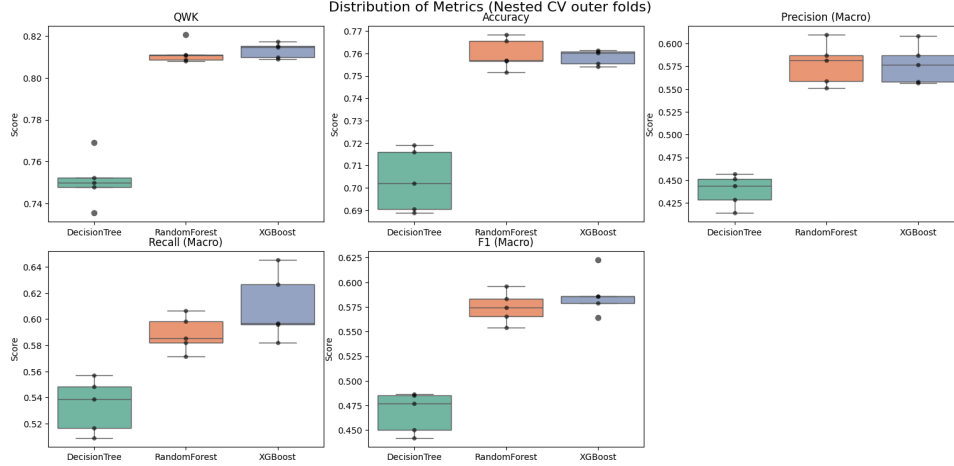
- **Outer Loop (5 Folds):** Evaluates true out-of-sample generalization across unobserved partitions.
- **Inner Loop (3 Folds):** Executes within each outer training fold to select optimal hyperparameters, PLS-DA components, and SMOTENC parameters without leakage.

Candidate architectures include Decision Trees (Baseline), Random Forest, and XGBoost. Table 2 summarizes cross-validated performance metrics across outer validation folds.

Figure 9 illustrates metric distributions across outer validation folds, confirming XGBoost’s dominance across QWK, Accuracy, and Macro F1 metrics.

Table 2. Comparison of candidate models (Nested Cross-Validation, Post-Release Mode).

Model	HP tuning	CV strategy	QWK (Val)	Macro F1
Baseline (Decision Tree)	Optuna / Grid Search	5×3 Nested CV	0.750	0.480
Random Forest	Optuna / Grid Search	5×3 Nested CV	0.811	0.575
XGBoost (Champion)	Optuna / Grid Search	5×3 Nested CV	0.817	0.585

**Figure 6.** Distribution of evaluation metrics across outer nested CV folds.

3.5 Hyperparameter Optimization

Two parallel optimization strategies were implemented to tune structural and tree hyperparameters:

1. **Strategy A (Two-Stage Bayesian Search):** Structural parameters (`max_tfidf_features=15`, `n_components=20`) were selected using a preliminary search across probe estimators. Hyperparameters were subsequently tuned using Optuna across continuous search spaces.
2. **Strategy B (Cached Unified Grid Search):** Joint optimization was validated via grid search using `joblib.Memory` disk caching. Ordering stable preprocessing transformers at the front minimized re-computation of TF-IDF, PLS-DA, and SMOTENC states.

Optuna tuning converged on the following optimal continuous parameters for XGBoost:

- `n_estimators`: 397, `max_depth`: 5
- `learning_rate`: ≈ 0.107 , `min_child_weight`: 5
- `subsample`: ≈ 0.965 , `reg_lambda`: ≈ 0.825

3.6 Statistical Significance Testing

Non-parametric tests evaluated performance differences across outer folds ($N = 5$):

- **Friedman Test:** Confirmed global statistically significant differences across model distributions ($p < 0.05$).
- **Wilcoxon Signed-Rank Test:** Pairwise testing between XGBoost and baselines is bounded at a minimum achievable two-tailed p-value of $p_{\min} = 2 \times (1/2^5) = 0.0625$ due to $N = 5$ outer folds. Because $p < 0.05$ is mathematically unachievable under $N = 5$, absolute fold dominance ($p = 0.0625$) across all 5 outer splits serves as the empirical indicator of statistical superiority.

3.7 Final Model Training & Calibration

The final XGBoost model was trained on the combined 80% training split. Post-hoc probability calibration was performed using Differential Evolution (DE) to optimize a 5-dimensional weight vector multiplying raw predicted class probabilities (`predict_proba`) against the Quadratic Weighted Kappa penalty function.

Generalization was evaluated on the held-out 20% test set ($N = 5,847$), which remained untouched during preprocessing and optimization.

After the evaluation, the final train used the entire dataset, in order to fit the model on all the available data.

4. Results and Discussion

4.1 Performance Evaluation

On the held-out test dataset, the calibrated XGBoost model achieved a final Quadratic Weighted Kappa of $QWK \approx 0.7026$ in Pre-Release mode and ≈ 0.840 in Post-Release mode.

Figure 7 shows the test set confusion matrices for Pre-Release and Post-Release operational modes following post-hoc calibration.

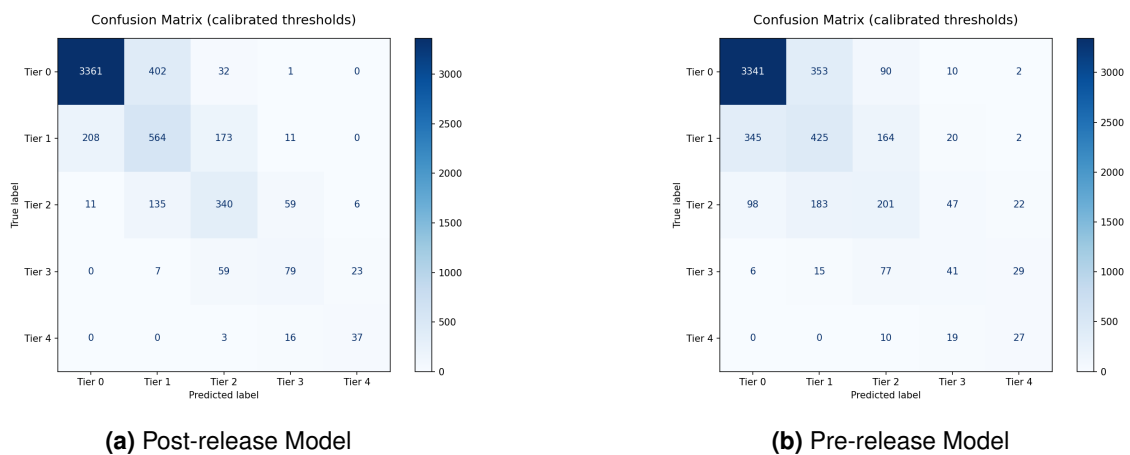


Figure 7. XGBoost Confusion Matrices.

Confusion matrix analysis demonstrates strong spatial clustering along the main diagonal. Severe off-diagonal misclassifications (e.g., predicting Tier 0 for a true Tier 4 game) are eliminated. Observed classification noise is confined strictly to adjacent classes (e.g., true Tier 1 titles predicted as Tier 0 or Tier 2). This boundary ambiguity represents the dataset's intrinsic predictive limit, as static metadata lacks visibility into unmodeled external market forces such as viral social media marketing, streamer adoption, and publisher ad budgets.

4.2 Model Explainability

Model transparency is established via an Inverse Scaling Mechanism that converts scaled numerical inputs back into real-world units (e.g., GB of RAM, USD price) before generating visual explanations.

Global feature attributions were extracted using SHAP TreeExplainer. Figure 8 presents SHAP Beeswarm plots for Tier 0 (Long-Tail) and Tier 4 (AAA Hit) in Pre-Release mode.

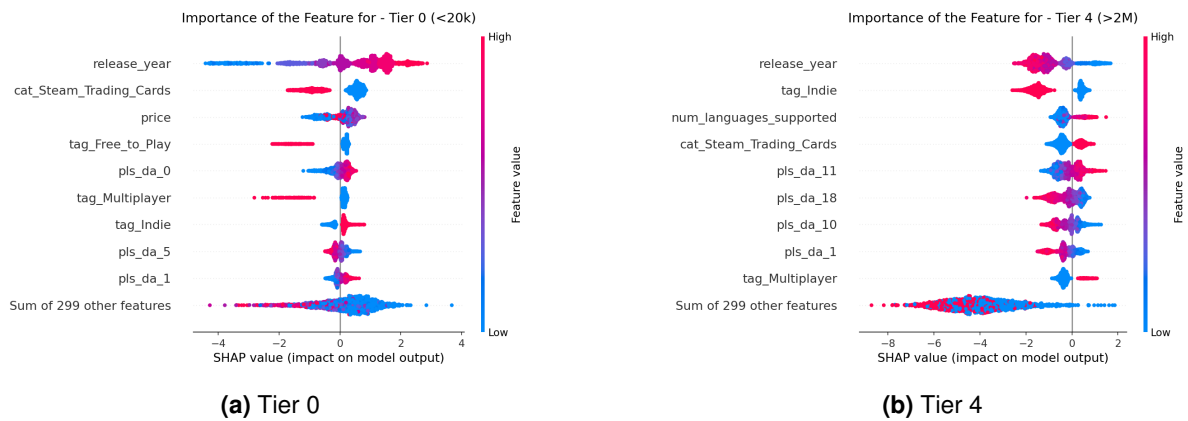


Figure 8. Global SHAP feature attributions for Tier 0 and Tier 4.

SHAP analysis reveals clear market signals:

- **Tier 4 Drivers:** High base pricing (for example $> \$19.99$), extensive language localization, and higher minimum RAM requirements push predictions toward Tier 4, acting as proxies for high-budget development.
- **Tier 0 Drivers:** Low pricing, lack of language localizations, and indie tags anchor predictions to Tier 0.

Local interpretability was implemented using LIME on representative test samples. Resampling pipeline states (Random UnderSampler, SMOTENC, Tomek Links) were replayed in isolated memory to construct background perturbation distributions, allowing developers to inspect specific feature thresholds driving individual game predictions.

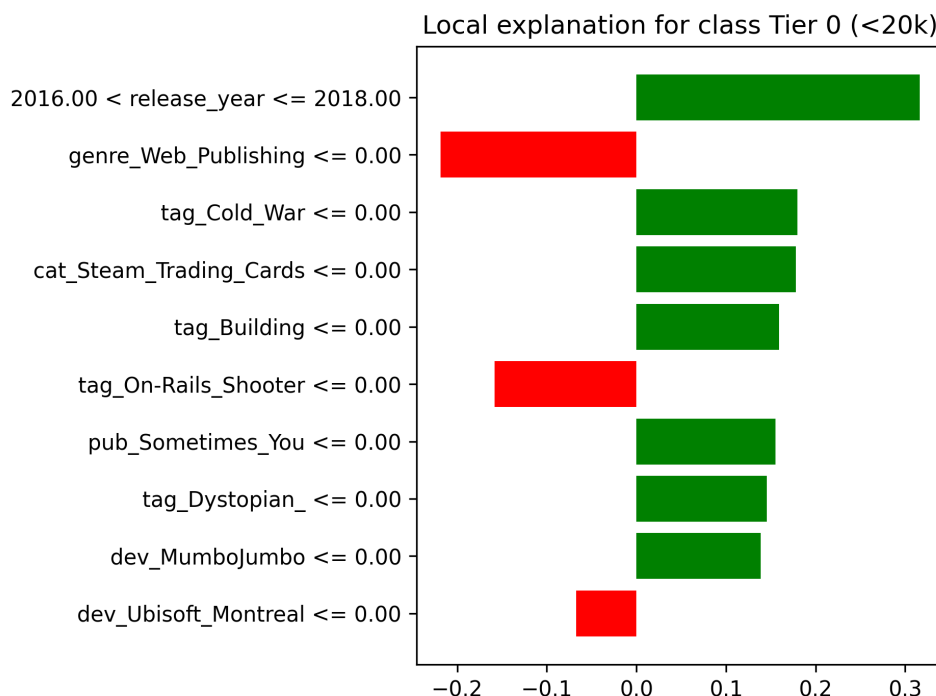


Figure 9. Local explanation case study generated via LIME about class 0.

More XAI charts are available in `results/post_release_model/xai_plots` and `results/pre_release_model/xai_plots`.

5. Conclusions

We presented HitPredictor-Steam, an end-to-end machine learning pipeline for predicting commercial video game adoption tiers on Steam using storefront metadata. By enforcing strict bifurcation between Pre-Launch and Post-Launch operational modes, the architecture guarantees zero data leakage, providing independent developers with a reliable day-zero forecasting tool.

Through dynamic majority undersampling, targeted SMOTENC minority synthesis (restricted to Tier 4), and Tomek Links boundary pruning, the pipeline effectively addresses severe power-law class imbalance. Supervised PLS-DA dimensionality reduction combined with dense SentenceTransformer embeddings (all-mpnet-base-v2) converts high-dimensional text fields into discriminative latent components. Hyperparameter optimization via Bayesian search and post-hoc probability calibration using Differential Evolution achieved a calibrated test performance of QWK ≈ 0.7026 . Inverse-scaled SHAP and LIME suites ensure complete transparency for industry stakeholders.

Limitations include lack of visibility into unmodeled extrinsic factors such as marketing spend and streamer adoption. Future work will extend this framework by implementing an auxiliary regression pipeline to predict community Approval Ratings (review_ratio), projecting game prototypes onto a 2D Market Fit Matrix (Volume vs. Sentiment) and integrating multimodal deep learning models to analyze promotional screenshot and trailer aesthetics.

References

- [1] Davis, N. (2019). Steam store games (clean dataset & raw data). Kaggle. <https://www.kaggle.com/datasets/nikdavis/steam-store-raw>
- [2] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP-IJCNLP*. <https://aclanthology.org/D19-1410/>
- [3] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *JAIR*, 16, 321–357. DOI: 10.1613/jair.953. <https://www.jair.org/index.php/jair/article/view/10302>
- [4] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *KDD*, pp. 2623–2631. <https://arxiv.org/abs/1907.10902>
- [5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD*, pp. 785–794. <https://arxiv.org/abs/1603.02754>
- [6] Storn, R., & Price, K. (1997). Differential evolution – a simple and efficient heuristic for global optimization. *Journal of Global Optimization*, 11(4), 341–359. <https://doi.org/10.1023/A:1008202821328>
- [7] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *NeurIPS*, 30, 4765–4774. <https://arxiv.org/abs/1705.07874>
- [8] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. *KDD*, pp. 1135–1144. <https://arxiv.org/abs/1602.04938>